



**Universitatea Tehnică a Moldovei Facultatea Calculatoare, Informatică și
Microelectronică**

Departamentul Inginerie Software și Automatică

BOTNARI MARIA-ELENA, FAF-232

REPORT

***Laboratory Work nr. 1
at Formal Languages & Finite Automata***

Checked by:
Dumitru Crețu, *university assistant*
DISA, FCIM, UTM

Chișinău – 2025

Theory

A formal grammar is a set of rules used to generate strings in a specific language. It consists of non-terminal symbols (which can be replaced), terminal symbols (which appear in the final string), production rules (which define how symbols are replaced), and a start symbol.

A finite automaton (FA) is a computational model used to recognize patterns in strings. It consists of states, transitions, a start state, and accepting states. After converting a grammar into an FA, we can verify if a given string belongs to the language defined by the grammar.

Objectives

- Implement a class to represent a grammar.
- Generate five valid strings based on the grammar rules.
- Convert the grammar into a finite automaton.
- Check if a given string can be derived using the finite automaton.

Implementation Description

Grammar Class: The Grammar class represents a formal grammar, storing its components:

- **Non-terminals (V_n)** – symbols that can be replaced.
- **Terminals (V_t)** – symbols that appear in the final string.
- **Production rules (P)** – rules that describe how non-terminals transform.
- **Start symbol (S)** – the symbol where the string generation begins.

```
pairs = self.P.split(", ")
for pair in pairs:
    a, b = pair.split("->")
    if "|" in b:
        rules = b.split("|")
    else:
        rules = [b]

    if a in self.P_dictionary:
        self.P_dictionary[a].extend(rules)
    else:
        self.P_dictionary[a] = rules
```

Fig. 1: Snippet of code from the Grammar Class

The production rules are stored in a dictionary, making it easy to find replacements. The constructor reads each rule, splits it into left-hand and right-hand sides, and stores them as items in the dictionary.

Generating Strings: A function generates valid strings by applying random production rules. The process starts with the start symbol and replaces non-terminals until only terminals remain. The function ensures that five unique strings are created. Each transformation step is recorded to show how the final string was formed.

```
while any(char in self.Vn for char in string):
    new_string = ""
    for char in string:
        if char in self.P_dictionary:
            new_string += random.choice(self.P_dictionary[char])
        else:
            new_string += char
    steps.append(f"{string} -> {new_string}")
    string = new_string
```

Fig. 2: Snippet of code from the generate_string function

Converting Grammar to FA: A separate FA class represents a **Finite Automaton**, converting the grammar into:

- **States** – represented by non-terminals.
- **Transitions** – describe how the automaton moves between states based on input symbols.
- **Accepting states** – where valid strings can end.
- **Start state** – taken from the grammar's start symbol.

The class extracts these elements from the grammar and builds a state-transition model.

Checking String in FA: A function was implemented to simulate state transitions while reading a string. It iterates over each character, showing how the FA transitions from one state to another. In the end, it checks if the final state is an accepting state, determining whether the string is valid.

```

for symbol in string:
    if (current_state, symbol) in transitions:
        next_state = transitions[(current_state, symbol)]
        steps.append(f"Read '{symbol}', move from {current_state} to {next_state}")
        current_state = next_state
    else:
        steps.append(f"Read '{symbol}', no transition exists. String rejected.")
        return steps

if current_state in accepting_states:
    steps.append(f"String accepted: Ended in accepting state {current_state}.")
else:
    steps.append(f"String rejected: Ended in non-accepting state {current_state}.")

```

Fig. 3: Snippet of code from the check_string_via_transition function

Results:

```

The steps taken to generate bcd: ['S -> bD', 'bD -> bcD', 'bcD -> bcd']
The steps taken to generate abdf: ['S -> aS', 'aS -> abD', 'abD -> abdR', 'abdR -> abdf']
The steps taken to generate fbf: ['S -> fR', 'fR -> fbR', 'fbR -> fbf']
The steps taken to generate bcdf: ['S -> bD', 'bD -> bcD', 'bcD -> bcdR', 'bcdR -> bcdf']
The steps taken to generate ff: ['S -> fR', 'fR -> ff']

5 Unique Strings: {'fbf', 'bcdf', 'bcd', 'abdf', 'ff'}

```

Fig. 4: The strings generated

```

Converting the object of type Grammar to type Finite Automaton
States: {'S', 'R', 'D'}
Transitions: {('S', 'a'): 'S', ('S', 'b'): 'D', ('S', 'f'): 'R', ('D', 'c'): 'D', ('D', 'd'): 'R', ('R', 'b'): 'R'}
Terminals: {'R', 'D'}
Start state: S

```

Fig. 5: The Finite Automaton object

```

Checking if the string aaaabd can be obtained via the state transition from FA:
['Start at S', "Read 'a', move from S to S", "Read 'a', move from S to S", "Read 'a', move from S to S", "Read 'a', move from S to S", "Read 'b', move from S to D", "Read 'd', move from D to R", 'String accepted: Ended in accepting state R.']

```

Fig. 6: The results after checking a string via transitions

Conclusion

In conclusion, this code successfully converts a formal grammar into a finite automaton and checks if a string belongs to the language. The Grammar class stores the grammar rules, generates valid strings, and tracks the transformation steps. The FA class builds a finite automaton using the grammar's states and transitions, allowing for structured string validation. By simulating state transitions, we can determine whether a given string follows the defined rules. This approach helped me understand how grammars and finite automata work together, making it easier to process and verify structured languages.